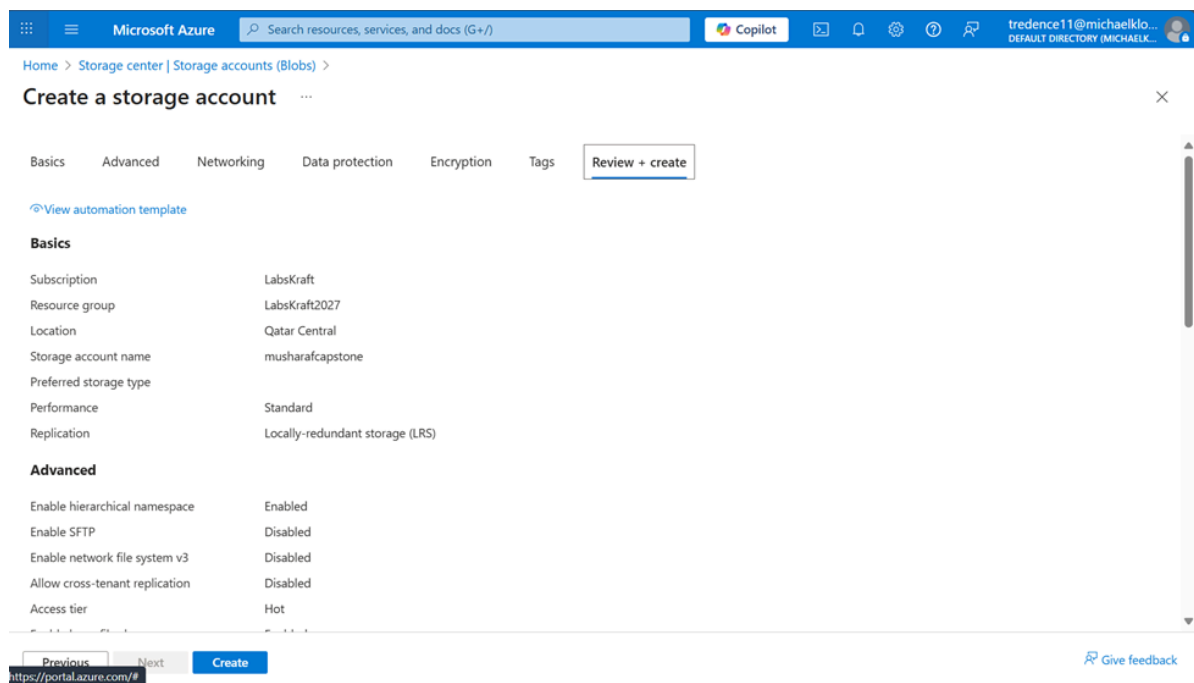


# OTT Analysis

StreamFlix, a growing over-the-top (OTT) streaming platform, needs to gain a deeper understanding of its users, content, and the reasons for subscriber churn. While they currently collect data from various sources like application logs, subscription systems, and ad servers, this data isn't unified or managed in a single, governed platform

## 1. Create a new storage account (musharafcapstone)



The screenshot displays the Microsoft Azure portal interface for creating a new storage account. The top navigation bar includes the Microsoft Azure logo, a search bar, and user information. The main heading is 'Create a storage account'. Below this, there are tabs for 'Basics', 'Advanced', 'Networking', 'Data protection', 'Encryption', and 'Tags'. The 'Basics' tab is active, showing the following configuration details:

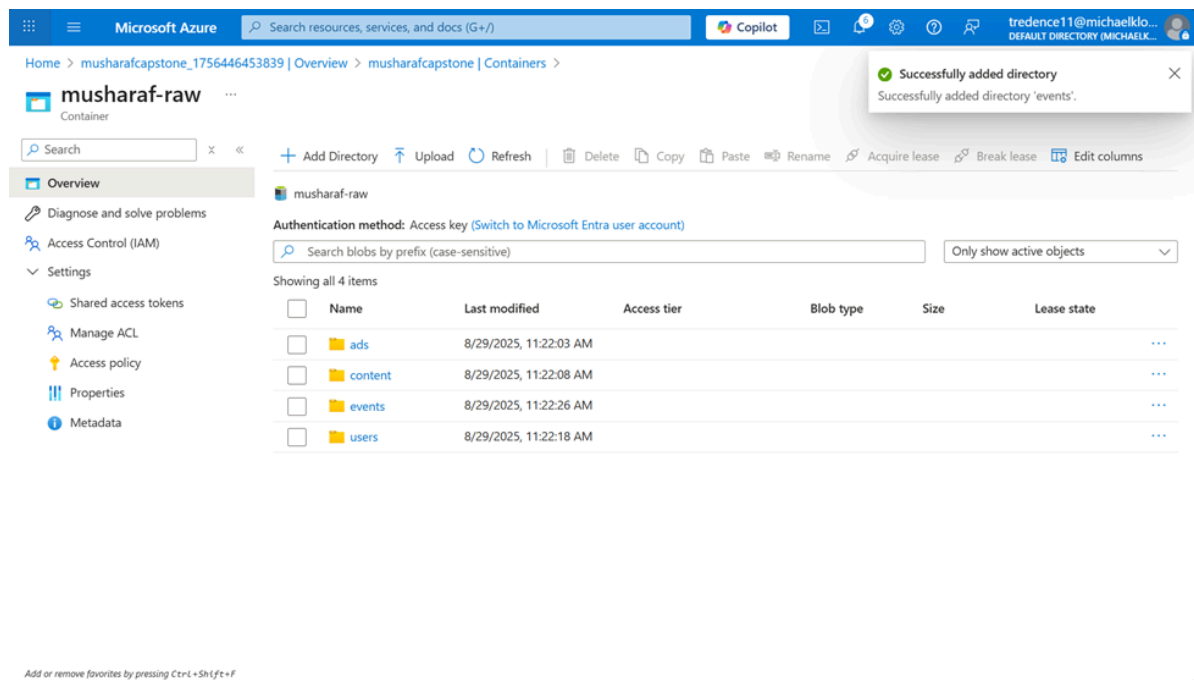
| Property               | Value                           |
|------------------------|---------------------------------|
| Subscription           | LabsKraft                       |
| Resource group         | LabsKraft2027                   |
| Location               | Qatar Central                   |
| Storage account name   | musharafcapstone                |
| Preferred storage type |                                 |
| Performance            | Standard                        |
| Replication            | Locally-redundant storage (LRS) |

Below the 'Basics' tab, the 'Advanced' tab is visible, showing the following configuration details:

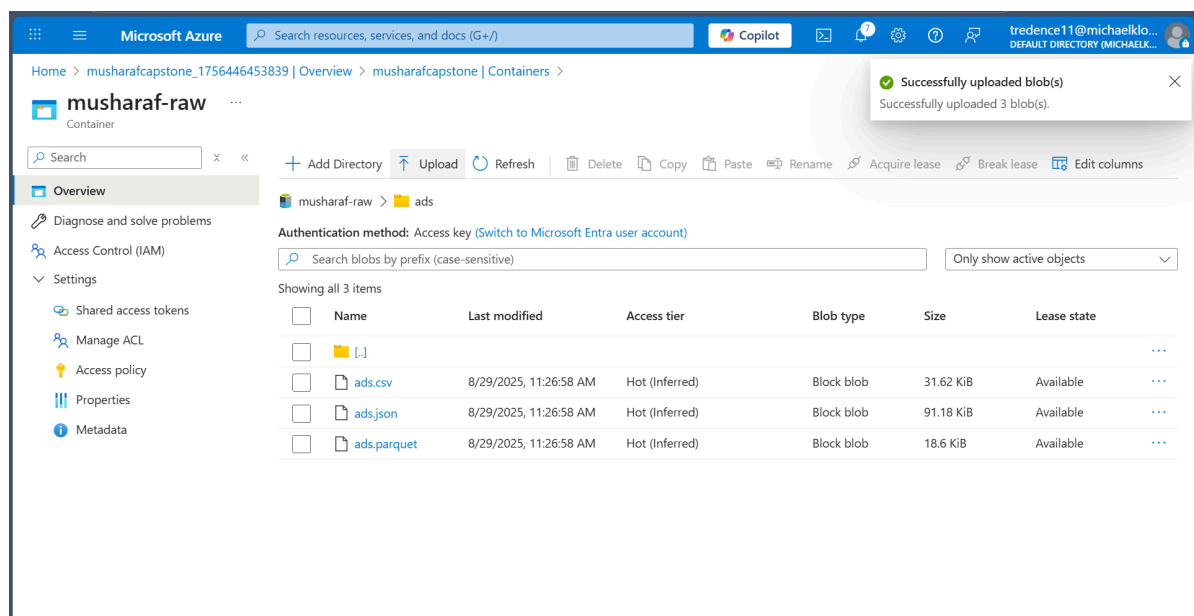
| Property                       | Value    |
|--------------------------------|----------|
| Enable hierarchical namespace  | Enabled  |
| Enable SFTP                    | Disabled |
| Enable network file system v3  | Disabled |
| Allow cross-tenant replication | Disabled |
| Access tier                    | Hot      |

At the bottom of the page, there are buttons for 'Previous', 'Next', and 'Create'. The 'Create' button is highlighted in blue. The URL in the address bar is <https://portal.azure.com/#>.

## 2. Now create a new container, and add new directories (*users, content, ad, event*)



### 3. After creating new containers, add the files in these directories (json|csv|parquet)



### 4. Now create a new Access Connector (musharaf\_connector\_OTT)

Home > AccessConnectorCreate-20250829114025 | Overview

Deployment

Search x << Delete Cancel Redeploy Download Refresh

Overview

Inputs

Outputs

Template

✓ Your deployment is complete

Deployment name : AccessConnectorCreate-20250... Start time : 8/29/2025, 11:40:55 AM  
Subscription : LabsKraft Correlation ID : 609651f7-e70b-4daf-a182-792...  
Resource group : LabsKraft2027

> Deployment details

✓ Next steps

Go to resource

Cost management

Get notified to stay within your budget and prevent unexpected charges on your bill.  
[Set up cost alerts >](#)

Microsoft Defender for Cloud

Secure your apps and infrastructure  
[Go to Microsoft Defender for Cloud >](#)

Free Microsoft tutorials

[Start learning today >](#)

Work with an expert

Azure experts are service provider partners who can help manage your assets on Azure and be your first line of support.  
[Find an Azure expert >](#)

<https://portal.azure.com/#home> Ctrl+Shift+F

- Now go to Storage Account > IAM Control > Add role assignment, and assign role to the Blob Storage

Home > musharafcapstone | Access Control (IAM) > Add role assignment

Role Members Conditions Review + assign

Role Storage Blob Data Contributor

Scope /subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourcegroups/LabsKraft2027/providers/Microsoft.Storage/storageAccounts/musharafcapstone

| Members | Name                   | Object ID                            | Type                                    |
|---------|------------------------|--------------------------------------|-----------------------------------------|
|         | musharaf_connector_OTT | a2827948-b0e8-44f6-9668-ba5cc371fcf5 | Access Connector for Azure Databricks ⓘ |

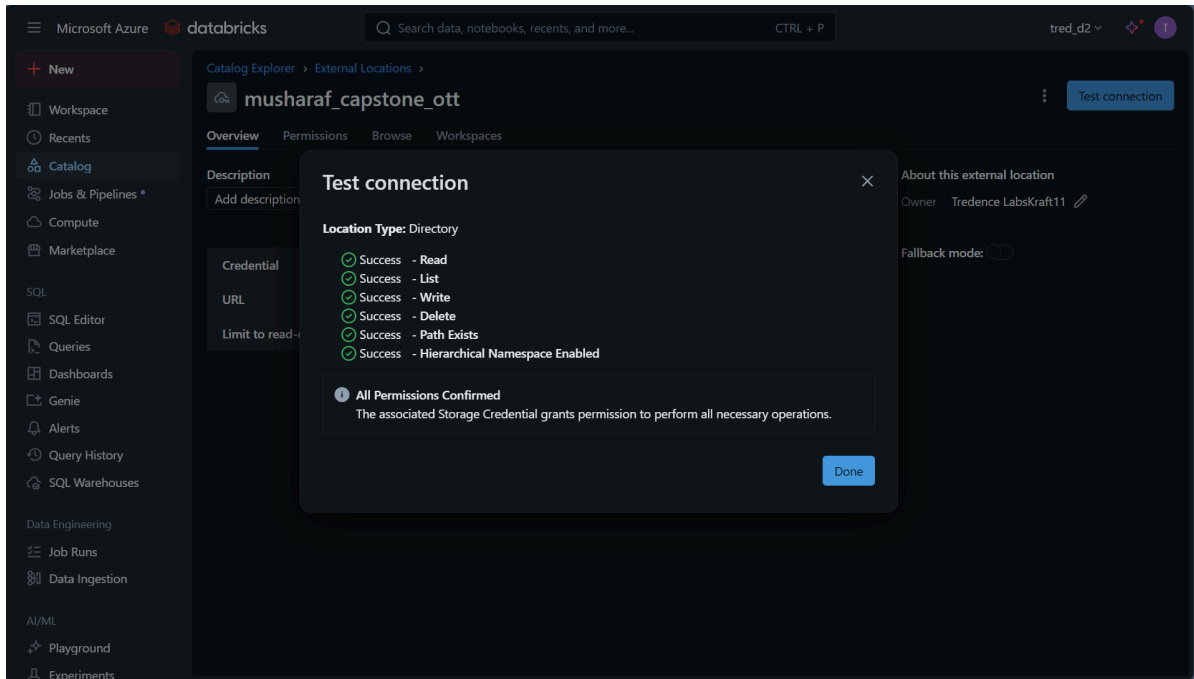
Description No description

Condition None

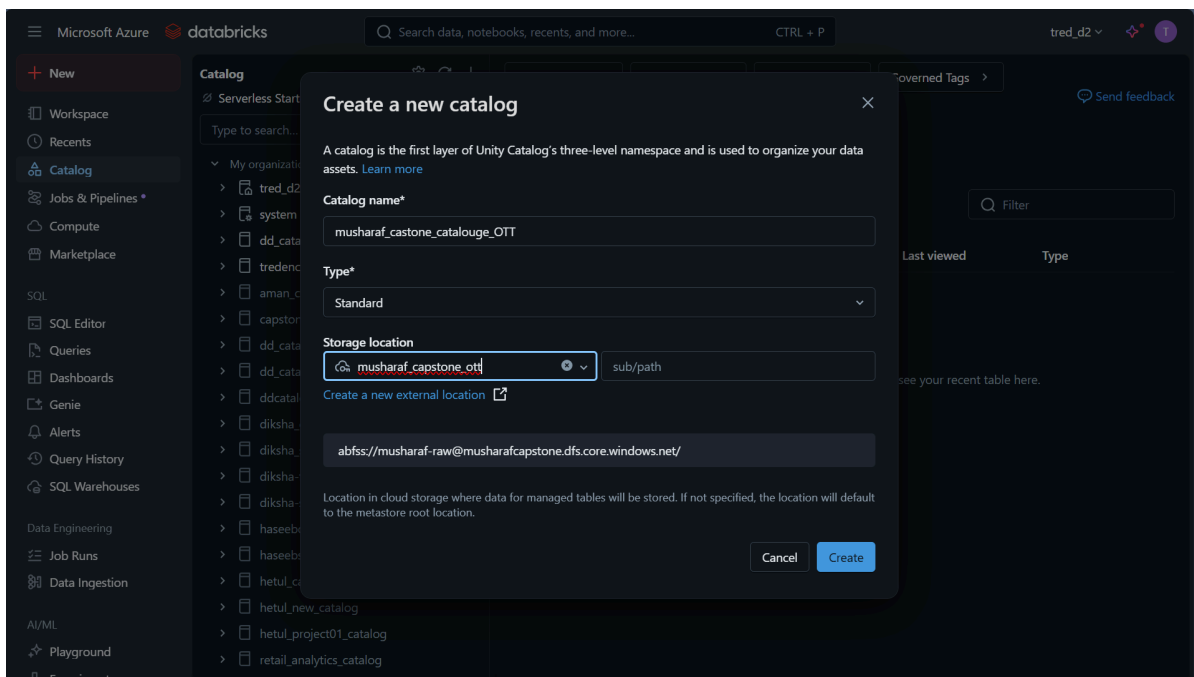
Review + assign Previous Next

Feedback

- Now go to Azure Databricks > catalogue> Add New catalogue > Create a new external location



7. Now go to Add New catalogue > and add the newly created ext location and create a new catalogue



8. Now Open the Data files (JSON/CSV/Parquest) and go through each of the files to understand the type of anomilies

- a. For users.csv 

- i. Missing Values: In user\_id, plan\_type or churn\_flag
  - ii. Invalid Data: INVALID\_FRB7 , INVALID\_8VO4 in plan\_type of billing\_cycle
  - iii. Data Type Issues: 48.421052631578945 like values in churn\_rate
- b. events.csv 📁:
- i. **Missing Data:** In start\_time and end\_time columns.
  - ii. **Invalid Data:** INVALID\_9NY3 and INVALID\_WS3K in user\_id and geo
  - iii. **Data Type Issues:** empty bitrate column
- c. ads.csv 📁
- **Missing Values:** In the impressions , clicks , and revenue columns
  - **Invalid Data:** INVALID\_ZDVY in ad\_id
  - **Outliers & Data Type Issues:** 10483.552631578948 ) in the click column.
- d. content.csv 📁
- **Missing Values:** in the content\_id column.
  - **Invalid Data:** INVALID\_2ATR , INVALID\_D83X in gener or region\_availability
  - **Outliers & Data Type Issues:** 10595.78947368421 in duration column

## 9. Now open a new Notebook and Create Bronze Delta table locations

```
bronze_path = "abfss://musharaf-raw@musharafcapstone.dfs.core.windows.net/bronze/"
```

```
users_bronze = (spark.read.format("csv").option("header", "true").load("abfss://musharaf-raw@musharafcapstone.dfs.core.windows.net/users/"))
ads_bronze = (spark.read.format("json").load("abfss://musharaf-raw@musharafcapstone.dfs.core.windows.net/ads/"))
content_bronze = (spark.read.format("parquet").load("abfss://musharaf-raw@musharafcapstone.dfs.core.windows.net/content/"))
```

```
users_bronze.write.format("delta").mode("overwrite").save(bronze_path +
```

```

"users")
ads_bronze.write.format("delta").mode("overwrite").save(bronze_path +
"ads")
content_bronze.write.format("delta").mode("overwrite").save(bronze_path
+ "content")

```

## Streaming data ingestion

```

event_hub_conf = {
    'eventhubs.connectionString': "Endpoint=sb://musharafcapstone.service
bus.windows.net/;SharedAccessKeyName=capstoneprj;SharedAccessKey
=;EntityPath=musharafcapstoneeventhub"
}

events_stream = (spark.readStream
    .format("eventhubs")
    .options(**event_hub_conf)
    .load())

from pyspark.sql.functions import from_json, col
from pyspark.sql.types import StructType, StructField, StringType, TimestampType

schema = StructType([
    StructField("event_id", StringType(), True),
    StructField("user_id", StringType(), True),
    StructField("content_id", StringType(), True),
    StructField("start_time", TimestampType(), True),
    StructField("end_time", TimestampType(), True),
    StructField("device", StringType(), True),
    StructField("geo", StringType(), True),
    StructField("bitrate", StringType(), True),
    StructField("error", StringType(), True)
])

parsed_events = events_stream.select(from_json(col("body").cast("string"), schema).as("parsed"))

```

```
g"), schema).alias("data")).select("data.*")

(parsed_events.writeStream
  .format("delta")
  .trigger(once=True)
  .option("checkpointLocation", bronze_path + "events/_checkpoint")
  .outputMode("append")
  .start(bronze_path + "events"))
```

## Silver Layer Cleaning

### Users Silver Table

- **Removed invalid rows:** Rows with null or empty `user_id` values were removed.
- **Masked user IDs:** The `user_id` column was masked with a SHA-256 hash to create `user_id_masked`.
- **Cleaned data:** Invalid values in `plan_type`, `billing_cycle`, and `payment_status` were replaced with `None`.
- **Casted data types:** The `churn_flag` column was cast to a boolean.
- **Removed duplicates:** Duplicate rows were removed

### Events Silver Table

- **Filtered data:** Rows with invalid `user_id` or missing `start_time` and `end_time` timestamps were removed.
- **Casted data types:** `bitrate` was cast to a double, and timestamps were cast to the `timestamp` data type.
- **Handled invalid values:** The `error` column was cleaned by replacing invalid values with `None`.
- **Derived a new metric:** A `watch_time_sec` column was calculated by finding the difference between `end_time` and `start_time`.
- **Removed duplicates:** Duplicate event records were removed.

- **Joined with other tables:** The table was joined with `users_silver` and `content_silver` to enrich the data.

## Ads Silver Table

- **Filtered data:** Rows with invalid `ad_id` values were removed.
- **Casted data types:** The `impressions` and `clicks` columns were cast to integers, and `revenue` was cast to a double.
- **Filtered outliers:** Click values greater than 10,000 were removed.
- **Removed duplicates:** Any duplicate ad records were removed.
- **Joined with events data:** The table was joined with `events_silver` on `content_id`.

## Content Silver Table

- **Filtered data:** Rows with null or empty `content_id` were removed.
- **Casted data types:** `duration_min` was cast to an integer.
- **Filtered outliers:** `duration_min` values outside the range of 1 to 500 were removed.
- **Handled invalid values:** Invalid values in the `genre` and `region_availability` columns were replaced with `None`.
- **Removed duplicates:** Duplicate content entries were removed.

```
from pyspark.sql.functions import sha2, col, when, lit, regexp_replace, trim, unix_timestamp, try_cast
```

```
users_silver = (
    users_bronze
    .where(col("user_id").isNotNull() & (trim(col("user_id")) != "")) # Remove rows with null or empty user_id
    .withColumn("user_id_masked", sha2(col("user_id"), 256))
    .withColumn("plan_type", when(regexp_replace(col("plan_type"), "INVALID_*", "").isNull() | (trim(col("plan_type")) == ""), lit(None)).otherwise(col("plan_
```



```

type)))
    .withColumn("billing_cycle", when(regexp_replace(col("billing_cycle"), "INVALID_.*", "").isNull() | (trim(col("billing_cycle")) == ""), lit(None)).otherwise(col("billing_cycle")))
    .withColumn("payment_status", when(regexp_replace(col("payment_status"), "INVALID_.*", "").isNull() | (trim(col("payment_status")) == ""), lit(None)).otherwise(col("payment_status")))
    .withColumn("churn_flag", try_cast(col("churn_flag"), "boolean")) # Handles float values like 1.0/0.0 correctly
    .dropDuplicates()
    .select("user_id_masked", "plan_type", "billing_cycle", "payment_status", "churn_flag")
)

users_silver.write.format("delta").mode("overwrite").save("abfss://musharaf-raw@musharafcystone.dfs.core.windows.net/silver/users")

```

- Events (Duplicates, clean, enrich with users + content)

```

from pyspark.sql.functions import unix_timestamp

events_silver = (
    events_bronze
    .where(col("user_id").isNotNull() & (trim(col("user_id")) != "") & ~col("user_id").like("INVALID_%"))
    .withColumn("bitrate", try_cast(col("bitrate"), "double"))
    .withColumn("error", when(regexp_replace(col("error"), "INVALID_.*", "").isNull() | (trim(col("error")) == ""), lit(None)).otherwise(col("error")))
    .withColumn("start_time", try_cast(col("start_time"), "timestamp"))
    .withColumn("end_time", try_cast(col("end_time"), "timestamp"))
    .where(col("start_time").isNotNull() & col("end_time").isNotNull()) # Drop rows with missing timestamps
    .withColumn("watch_time_sec", unix_timestamp("end_time") - unix_timestamp("start_time"))
    .dropDuplicates()
)

```

```

.join(users_silver, events_bronze.user_id == users_silver.user_id_masked, "left") # Join on the raw user_id
.join(content_silver, "content_id", "left")
)

events_silver.write.format("delta").mode("overwrite").save("abfss://musharaf-raw@musharafcapstone.dfs.core.windows.net/silver/events")

```

### Ads (Clean + Join)

```

ads_silver = (
    ads_bronze
    .where(col("ad_id").isNotNull() & (trim(col("ad_id")) != "") & ~col("ad_id").like("INVALID_%"))
    .withColumn("impressions", try_cast(col("impressions"), "int"))
    .withColumn("clicks", try_cast(col("clicks"), "int"))
    .where(col("clicks").between(0, 10000)) # Filter out click outliers like 10483.55
    .withColumn("revenue", try_cast(col("revenue"), "double"))
    .dropDuplicates()
    .join(events_silver, "content_id", "left")
)

ads_silver.write.format("delta").mode("overwrite").save("abfss://musharaf-raw@musharafcapstone.dfs.core.windows.net/silver/ads")

```

### Content (Minor Clean)

```

content_silver = (
    content_bronze
    .where(col("content_id").isNotNull() & (trim(col("content_id")) != ""))
    .withColumn("duration_min", try_cast(col("duration_min"), "int")) # Safe casting for numeric values
    .where(col("duration_min").between(1, 500)) # Filter out outlier durations like 10595.78
    .withColumn("genre", when(regex_replace(col("genre"), "INVALID_.*",

```

```

"").isNull() | (trim(col("genre")) == ""), lit(None)).otherwise(col("genre")))
    .withColumn("region_availability", when(regexp_replace(col("region_ava
ilability"), "INVALID_.*", "").isNull() | (trim(col("region_availability")) == ""),
    lit(None)).otherwise(col("region_availability")))
    .dropDuplicates()
)

content_silver.write.format("delta").mode("overwrite").save("abfss://mush
araf-raw@musharafcapstone.dfs.core.windows.net/silver/content")

```

## Gold Layer Cleaning

### a) Real-Time KPIs (concurrent viewers)

```

from pyspark.sql.functions import window, countDistinct

concurrent_viewers = (
    events_silver
    .groupBy(window("start_time", "1 minute"), "content_id", "device")
    .agg(countDistinct("user_id_masked").alias("concurrent_users"))
)

concurrent_viewers.write.format("delta").mode("overwrite").save("abfss://mu
sharaf-raw@musharafcapstone.dfs.core.windows.net/gold/concurrent_viewer
s")

```

### b) Historical Analytics

```

engagement_kpis = (
    events_silver
    .groupBy("genre", "plan_type", "device")
    .agg(
        avg("watch_time_sec").alias("avg_watch_time"),
        sum("watch_time_sec").alias("total_watch_time")
    )
)

```

```
engagement_kpis.write.format("delta").mode("overwrite").save("abfss://musharaf-raw@musharafcapstone.dfs.core.windows.net/gold/engagement_kpis")
```

### c) Ad Monetization

```
revenue_kpis = (  
    ads_silver  
    .groupBy("content_id")  
    .agg(  
        sum("revenue").alias("total_revenue"),  
        sum("impressions").alias("total_impressions"),  
        sum("clicks").alias("total_clicks")  
    )  
)
```

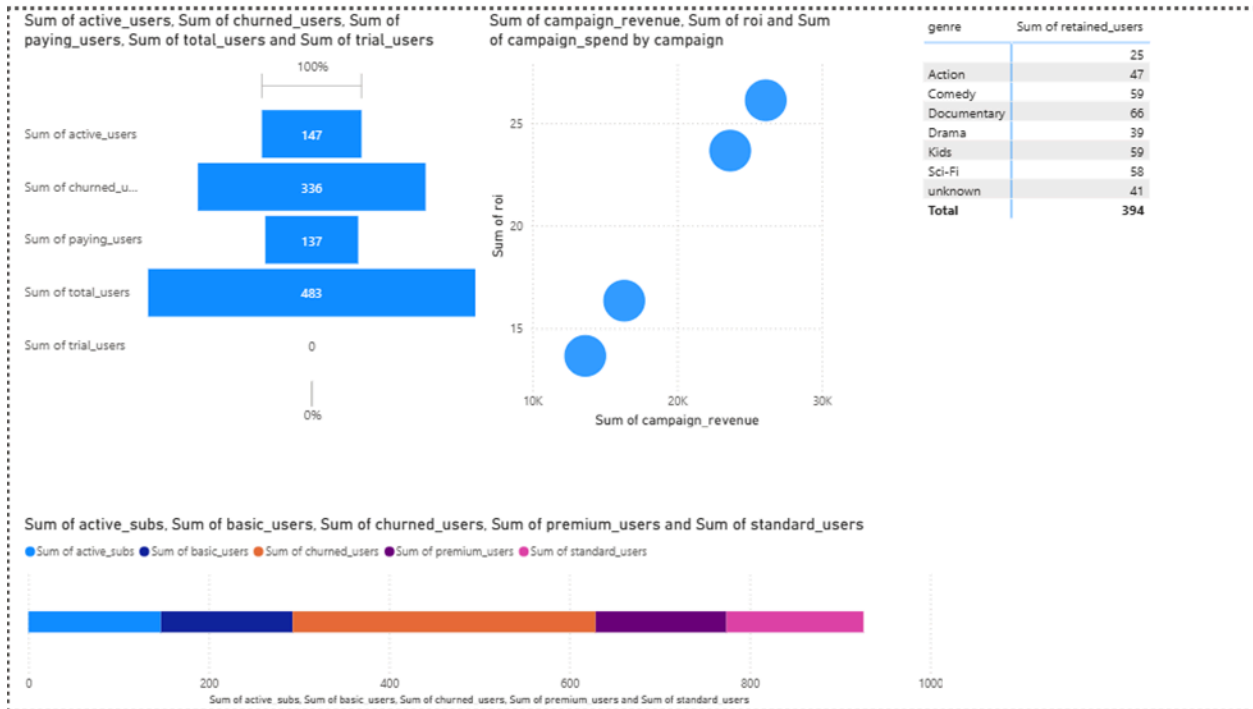
```
revenue_kpis.write.format("delta").mode("overwrite").save("abfss://musharaf-raw@musharafcapstone.dfs.core.windows.net/gold/revenue_kpis")
```

### d) Churn Feature Store

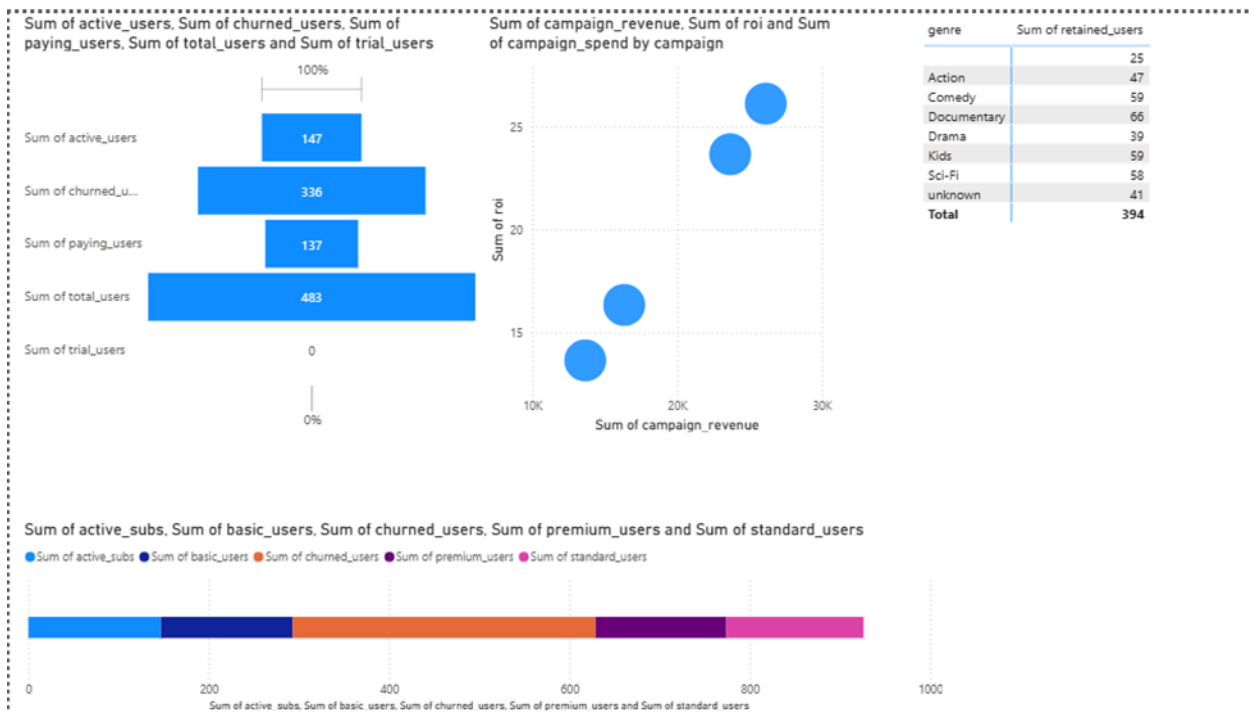
```
churn_features = (  
    events_silver  
    .groupBy("user_id_masked", "plan_type", "billing_cycle")  
    .agg(  
        sum("watch_time_sec").alias("total_watch_time"),  
        countDistinct("content_id").alias("unique_contents_watched"),  
        avg("bitrate").alias("avg_bitrate"),  
        max("churn_flag").alias("churn_flag")  
    )  
)
```

```
churn_features.write.format("delta").mode("overwrite").save("abfss://musharaf-raw@musharafcapstone.dfs.core.windows.net/gold/churn_features")
```

### Power Bi Notebook



| genre        | Sum of retained_users |
|--------------|-----------------------|
| Action       | 25                    |
| Comedy       | 47                    |
| Documentary  | 59                    |
| Drama        | 66                    |
| Kids         | 39                    |
| Sci-Fi       | 59                    |
| unknown      | 58                    |
| <b>Total</b> | <b>394</b>            |



| genre        | Sum of retained_users |
|--------------|-----------------------|
| Action       | 25                    |
| Comedy       | 47                    |
| Documentary  | 59                    |
| Drama        | 66                    |
| Kids         | 39                    |
| Sci-Fi       | 59                    |
| unknown      | 58                    |
| <b>Total</b> | <b>394</b>            |

Sum of total\_ad\_revenue, Sum of total\_revenue and Sum of total\_subscription\_revenue by plan\_type

